

Name: Prachi gohil

USN No.: 1AUA23BCS142

Task 1: Reading data from files

```
In [1]: import matplotlib.pyplot as plt
import pandas as pd
```

```
In [5]: file_path = 'supermarket_sales.csv'
df = pd.read_csv(file_path)

print(df.head(7))
```

	Invoice ID	Branch	City	Customer type	Gender	\
0	750-67-8428	A	Yangon	Member	Female	
1	226-31-3081	C	Naypyitaw	Normal	Female	
2	631-41-3108	A	Yangon	Normal	Male	
3	123-19-1176	A	Yangon	Member	Male	
4	373-73-7910	A	Yangon	Normal	Male	
5	699-14-3026	C	Naypyitaw	Normal	Male	
6	355-53-5943	A	Yangon	Member	Female	

	Product line	Unit price	Quantity	Tax 5%	Total	Date	\
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	
4	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019	
5	Electronic accessories	85.39	7	29.8865	627.6165	3/25/2019	
6	Electronic accessories	68.84	6	20.6520	433.6920	2/25/2019	

	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	10:29	Cash	76.40	4.761905	3.8200	9.6
2	13:23	Credit card	324.31	4.761905	16.2155	7.4
3	20:33	Ewallet	465.76	4.761905	23.2880	8.4
4	10:37	Ewallet	604.17	4.761905	30.2085	5.3
5	18:30	Ewallet	597.73	4.761905	29.8865	4.1
6	14:36	Ewallet	413.04	4.761905	20.6520	5.8

```
In [6]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Invoice ID             1000 non-null   object
1   Branch                 1000 non-null   object
2   City                   1000 non-null   object
3   Customer type          1000 non-null   object
4   Gender                 1000 non-null   object
5   Product line           1000 non-null   object
6   Unit price             1000 non-null   float64
7   Quantity               1000 non-null   int64
8   Tax 5%                 1000 non-null   float64
9   Total                  1000 non-null   float64
10  Date                   1000 non-null   object
11  Time                   1000 non-null   object
12  Payment                1000 non-null   object
13  cogs                   1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income           1000 non-null   float64
16  Rating                 1000 non-null   float64
dtypes: float64(7), int64(1), object(9)
memory usage: 132.9+ KB

```

In [7]: `df.shape`

Out[7]: (1000, 17)

Task 1.2: Observing and describing Data

In [8]: `df.describe()`

Out[8]:

	Unit price	Quantity	Tax 5%	Total	cogs	gross margin percentage	
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	10
mean	55.672130	5.510000	15.379369	322.966749	307.58738	4.761905	
std	26.494628	2.923431	11.708825	245.885335	234.17651	0.000000	
min	10.080000	1.000000	0.508500	10.678500	10.17000	4.761905	
25%	32.875000	3.000000	5.924875	124.422375	118.49750	4.761905	
50%	55.230000	5.000000	12.088000	253.848000	241.76000	4.761905	
75%	77.935000	8.000000	22.445250	471.350250	448.90500	4.761905	
max	99.960000	10.000000	49.650000	1042.650000	993.00000	4.761905	

In [13]: `df.head()`

Out[13]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085

In [11]: `df.tail()`

Out[11]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.01
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.59
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.29
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.91

Task 1.3: Selecting Columns from dataframe

In [14]: `print(df[['Branch', 'City', 'Gender', 'Product line']])`

	Branch	City	Gender	Product line
0	A	Yangon	Female	Health and beauty
1	C	Naypyitaw	Female	Electronic accessories
2	A	Yangon	Male	Home and lifestyle
3	A	Yangon	Male	Health and beauty
4	A	Yangon	Male	Sports and travel
..	...	...	...	...
995	C	Naypyitaw	Male	Health and beauty
996	B	Mandalay	Female	Home and lifestyle
997	A	Yangon	Male	Food and beverages
998	A	Yangon	Male	Home and lifestyle
999	A	Yangon	Female	Fashion accessories

[1000 rows x 4 columns]

Task 1.4: Add new columns to a DataFrame

```
In [26]: df['Discounted_Total'] = df['Total']*0.95
```

```
In [27]: df[['Discounted_Total']].head()
```

```
Out[27]:
```

	Discounted_Total
0	521.522925
1	76.209000
2	323.499225
3	464.595600
4	602.659575

Task 1.5: Applying functions on DataFrame

Task 6: Applying functions on multiple columns

```
In [28]: df['Discounted_Price'] = df['Total'] - df['Discounted_Total']

print(df.head(4))
```

	Invoice ID	Branch	City	Customer type	Gender	\
0	750-67-8428	A	Yangon	Member	Female	
1	226-31-3081	C	Naypyitaw	Normal	Female	
2	631-41-3108	A	Yangon	Normal	Male	
3	123-19-1176	A	Yangon	Member	Male	

	Product line	Unit price	Quantity	Tax 5%	Total	Date	\
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019	
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019	
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019	
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019	

	Time	Payment	cogs	gross margin percentage	gross income	Rating	\
0	13:08	Ewallet	522.83	4.761905	26.1415	9.1	
1	10:29	Cash	76.40	4.761905	3.8200	9.6	
2	13:23	Credit card	324.31	4.761905	16.2155	7.4	
3	20:33	Ewallet	465.76	4.761905	23.2880	8.4	

	Discounted_Total	Discounted_Price
0	521.522925	27.448575
1	76.209000	4.011000
2	323.499225	17.026275
3	464.595600	24.452400

In [29]: df

Out[29]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.14
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.21
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.28
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.20
...	...	...	...	...	...	...	...	...	...
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.01
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.59
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.29
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.91

1000 rows × 19 columns



Task 7: Deleting columns from DataFrame

```
In [30]: df = df.drop("Discounted_Total", axis = 1)
```

```
In [31]: df
```

Out[31]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.14
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.82
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.21
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.28
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.20
...	...	...	...	...	...	...	...	...	...
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2.01
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.59
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3.29
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30.91

1000 rows × 18 columns



Task 1.8: Writing a DataFrame to a file

```
In [20]: df.to_csv("lab1.csv")
```

Lecture 2

Task 2.1 Plot 1: Customize Font and Color

Task 2.2 Plot 2: Text Wrapping

Task 2.3 Plot 3: Positioning and Spacing

Task 2.4 Plot 4: Background and Box

Task 2.5 Plot 1,4: Customizing Spines

Task 2.6 Change Background Color of Plot 3

Task 2.7 Plot 1 a box with coords (2,2), (2,5), (5,2), (5,5)

Task 2.8 Plot 2 Make a circle with a plus

```
In [37]: import matplotlib.pyplot as plt
import numpy as np
from matplotlib.ticker import AutoMinorLocator, FormatStrFormatter

fig, ax = plt.subplots(2, 2, figsize=(10, 8))

# ----- Plot 1 -----
ax[0,0].set_title("Table 1 title", fontsize=14)
ax[0,0].set_xlabel("Table 1", color='red', fontsize=18)
ax[0,0].set_ylabel("Y label")

ax[0,0].plot([2,2,5,5,2],[2,5,5,2,2], linewidth=2)

ax[0,0].set_xlim(0.1,10)
ax[0,0].set_ylim(0,10)

ax[0,0].set_xscale('log')

ax[0,0].grid(True, linestyle='dotted', color='red')

ax[0,0].axhline(3)
ax[0,0].axvline(1)

ax[0,0].spines['top'].set_linewidth(2)
ax[0,0].spines['right'].set_linewidth(2)
ax[0,0].spines['left'].set_linewidth(2)
ax[0,0].spines['bottom'].set_linewidth(2)

# ----- Plot 2 -----
ax[0,1].set_title("Plot 2 Title")
ax[0,1].set_xlabel("X-axis for Plot 2")
ax[0,1].set_ylabel("Y-axis for Plot 2")

r = 3
theta = np.linspace(0, 2*np.pi, 400)
x = r * np.cos(theta)
```



```

y = r * np.sin(theta)

ax[0,1].plot(x, y)
ax[0,1].plot([-3, 3], [0, 0])
ax[0,1].plot([0, 0], [-3, 3])

ax[0,1].set_xlim(-10,10)
ax[0,1].set_ylim(-10,10)

# ----- Plot 3 -----
ax[1,0].set_title(
    "Plot 3 Title: Custom Positioning",
    color='white',
    backgroundcolor='purple'
)

ax[1,0].set_xlabel("X-axis for Plot 3")
ax[1,0].set_ylabel("Y-axis for Plot 3")

t = np.linspace(0, 4*np.pi, 400)
x = t * np.cos(t)
y = t * np.sin(t)

ax[1,0].plot(x,y)
ax[1,0].set_xlim(-10,10)
ax[1,0].set_ylim(-10,10)

ax[1,0].invert_xaxis()
ax[1,0].invert_yaxis()
ax[1,0].set_facecolor('lightgray')
ax[1,0].grid(True)

# ----- Plot 4 -----
ax[1,1].set_title("Plot 4 Title: Background and Box")

ax[1,1].set_xlabel(
    "X_label for Plot 4",
    color='black',
    backgroundcolor='red'
)

ax[1,1].set_ylabel("Y_label for Plot 4")

ax[1,1].plot([0,4,2,0],[0,0,3,0])

ax[1,1].axhline(0)
ax[1,1].axvline(0)

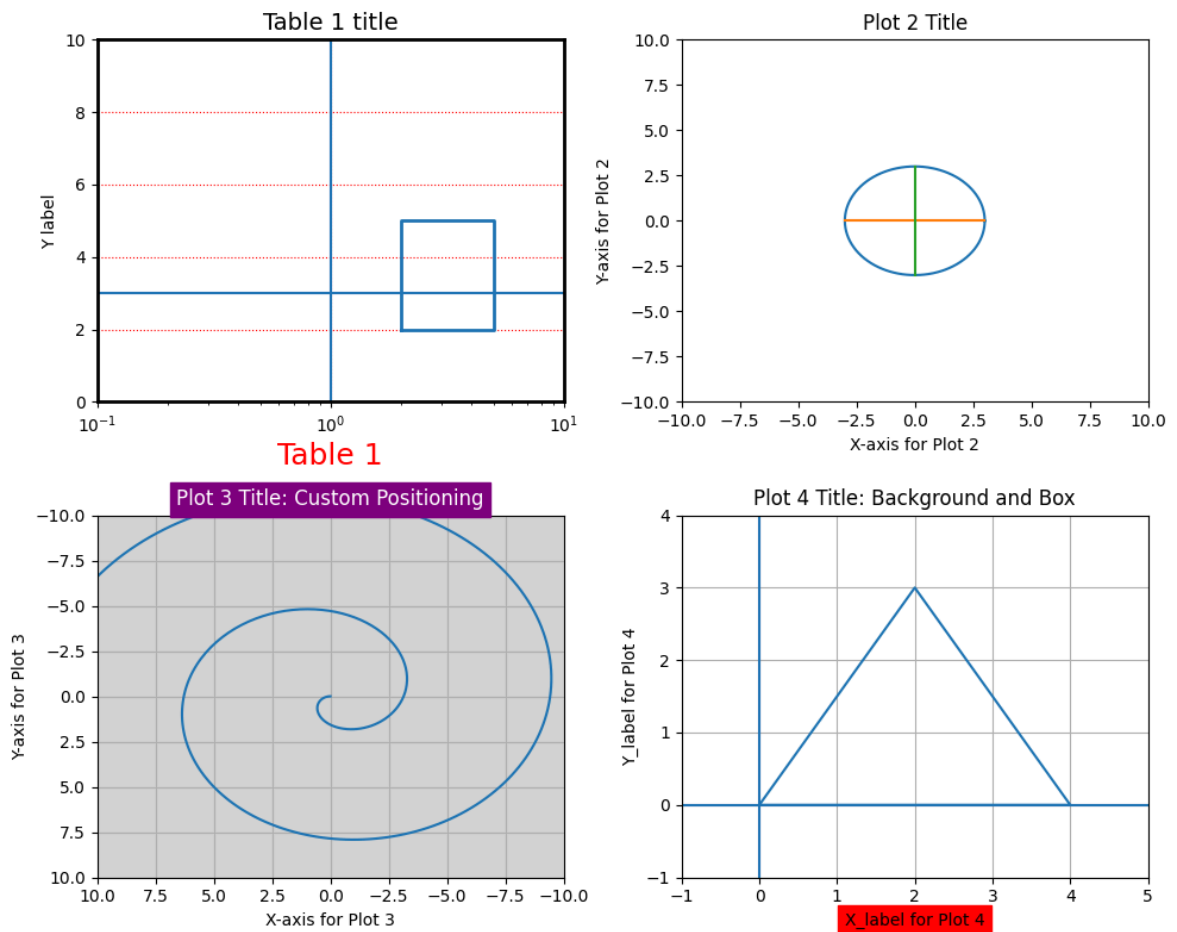
ax[1,1].set_xlim(-1,5)
ax[1,1].set_ylim(-1,4)

ax[1,1].set_xticks(np.arange(-1,6,1))
ax[1,1].set_yticks(np.arange(-1,5,1))

ax[1,1].grid(True)

plt.tight_layout()
plt.show()

```



Task 2.9 : Plotting Rectangle, Circle, Ellipse, Polygon using

matplotlib.patches

```
In [48]: import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, Circle, Polygon, Ellipse

fig, ax = plt.subplots(2, 2, figsize=(8,8))

# ----- Plot 1 : Rectangle -----
r1 = Rectangle(
    (1, 1),
    width=3,
    height=2,
    angle=30,
    facecolor='skyblue',
    edgecolor='navy',
    linewidth=2
)

ax[0,0].add_patch(r1)
ax[0,0].set_title("Rectangle Patch")
ax[0,0].set_xlim(0, 6)
ax[0,0].set_ylim(0, 6)
ax[0,0].set_aspect('equal')
ax[0,0].set_xlabel("X Axis")
ax[0,0].set_ylabel("Y Axis")
ax[0,0].grid(True)
```

```

# ----- Plot 2 : Circle -----
c1 = Circle(
    (3, 3),
    radius=1.5,
    facecolor='pink',
    edgecolor='purple',
    linewidth=2
)

ax[0,1].add_patch(c1)
ax[0,1].set_title("Circle Patch")
ax[0,1].set_xlim(0, 6)
ax[0,1].set_ylim(0, 6)
ax[0,1].set_aspect('equal')
ax[0,1].set_xlabel("X Axis")
ax[0,1].set_ylabel("Y Axis")
ax[0,1].grid(True)

# ----- Plot 3 : Ellipse -----
e1 = Ellipse(
    (3, 3),
    width=4,
    height=2,
    angle=45,
    facecolor='lightgreen',
    edgecolor='darkgreen',
    linewidth=2
)

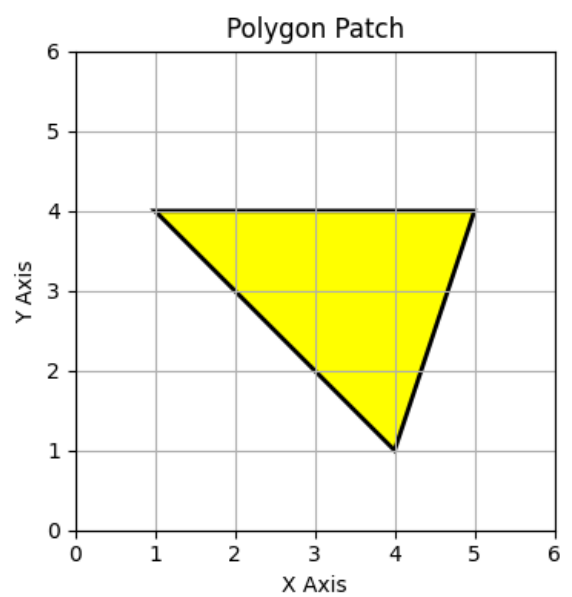
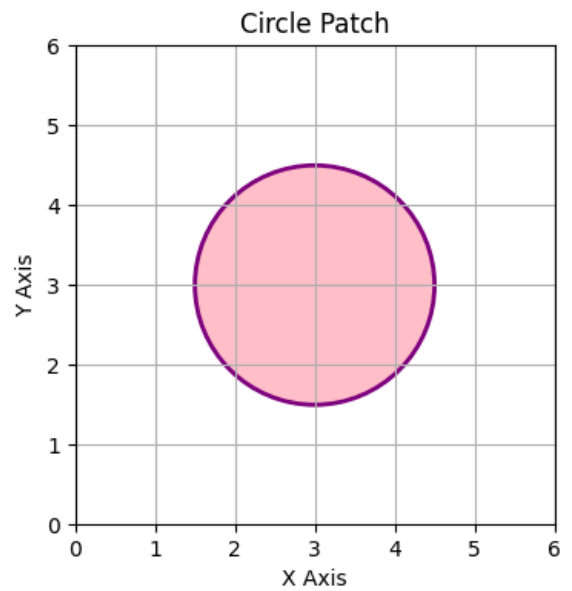
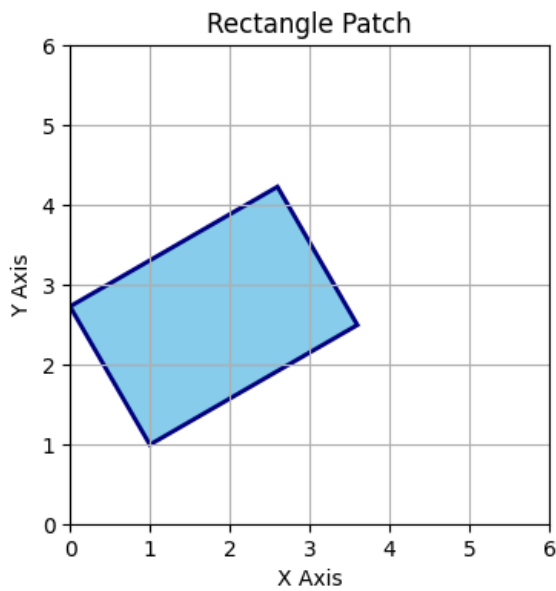
ax[1,0].add_patch(e1)
ax[1,0].set_title("Ellipse Patch")
ax[1,0].set_xlim(0, 6)
ax[1,0].set_ylim(0, 6)
ax[1,0].set_aspect('equal')
ax[1,0].set_xlabel("X Axis")
ax[1,0].set_ylabel("Y Axis")
ax[1,0].grid(True)

# ----- Plot 4 : Left Triangle -----
triangle = Polygon(
    [[4,1], [5,4], [1,4]], # Left pointing triangle
    closed=True,
    facecolor='yellow',
    edgecolor='black',
    linewidth=2
)

ax[1,1].add_patch(triangle)
ax[1,1].set_title("Polygon Patch")
ax[1,1].set_xlim(0, 6)
ax[1,1].set_ylim(0, 6)
ax[1,1].set_aspect('equal')
ax[1,1].set_xlabel("X Axis")
ax[1,1].set_ylabel("Y Axis")
ax[1,1].grid(True)

plt.tight_layout()
plt.show()

```

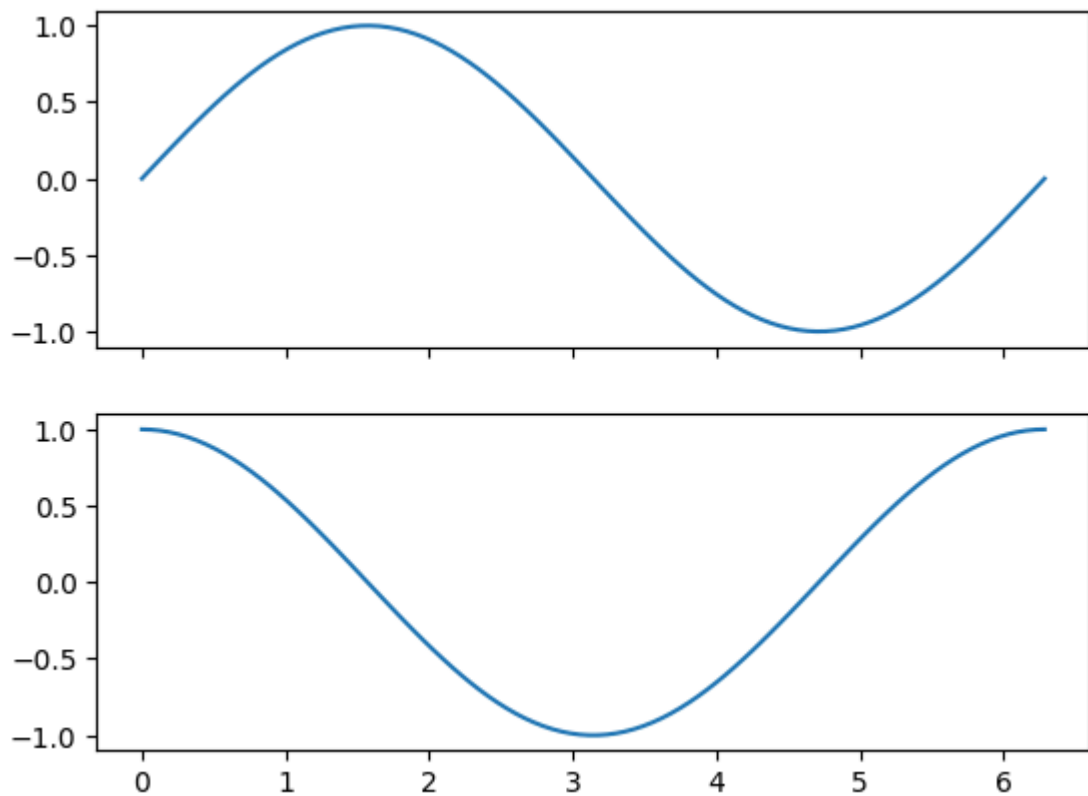


Task 2.10 Sharex vs twinx

```
In [49]: fig, ax = plt.subplots(2, 1, sharex=True)
x = np.linspace(0, 2*np.pi, 100)

ax[0].plot(x, np.sin(x))
ax[1].plot(x, np.cos(x))
```

```
Out[49]: [<matplotlib.lines.Line2D at 0x259541433d0>]
```



```
In [61]: import numpy as np
import matplotlib.pyplot as plt

x1 = np.linspace(1,5,5)
y1 = [1,3,2,6,7]
y2 = [6,2,3,1,0]

x2 = np.linspace(0,10,100)
s = np.sin(x2)
c = np.cos(x2)

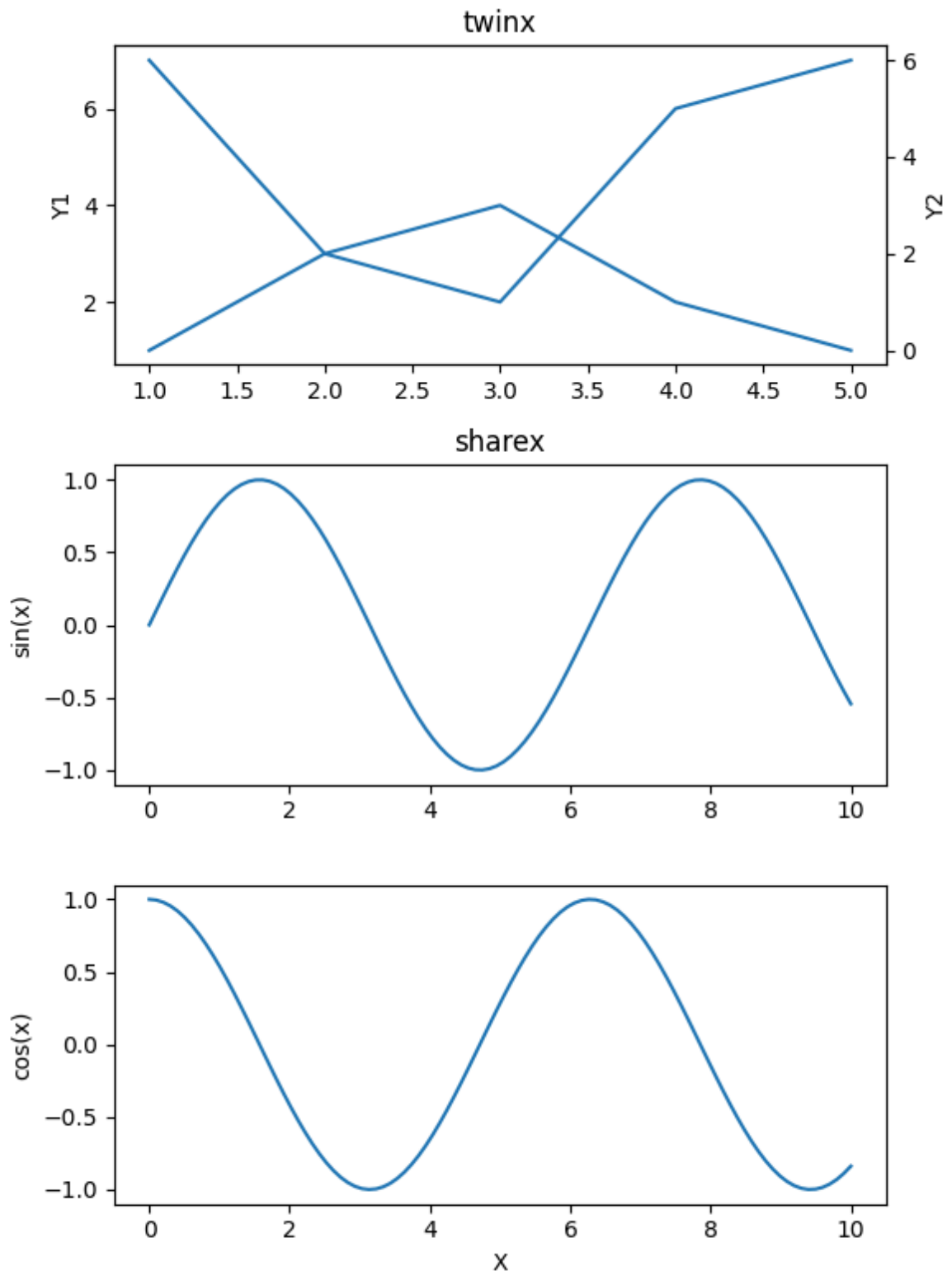
fig, ax = plt.subplots(3,1,figsize=(6,8))

# twinx
ax[0].plot(x1, y1)
ax[0].set_ylabel("Y1")
ax0b = ax[0].twinx()
ax0b.plot(x1, y2)
ax0b.set_ylabel("Y2")
ax[0].set_title("twinx")

# sharex part
ax[1].plot(x2, s)
ax[1].set_ylabel("sin(x)")
ax[1].set_title("sharex")

ax[2].plot(x2, c)
ax[2].set_ylabel("cos(x)")
ax[2].set_xlabel("X")

plt.tight_layout()
plt.show()
```



Task 2.11: Inset plotting (plot + legend + title)

```
In [62]: from mpl_toolkits.axes_grid1.inset_locator import inset_axes, mark_inset
import matplotlib.pyplot as plt
import numpy as np

# Sample data (you can replace this with your own)
x = np.linspace(40, 55, 300)
y = np.sin(x)

fig, ax = plt.subplots()
```

```

# Main plot
ax.plot(x, y)
ax.set_ylabel("signal amplitude")
ax.set_title("Zoomed plot")
ax.grid(True, linestyle="--", alpha=0.75)

# Zoom region
x1, x2 = 46.8, 48.3
y1, y2 = -0.4, 0.3

ax.axvline(x1, color="red")
ax.axvline(x2, color="red")
ax.axvspan(x1, x2, alpha=0.1)

# Inset axes (zoom box)
axins = inset_axes(
    ax,
    width="35%",
    height="45%",
    loc="upper right",
    borderpad=1.2
)

# Plot inside inset
axins.plot(x, y)
axins.set_xlim(x1, x2)
axins.set_ylim(y1, y2)
axins.grid(True, linestyle="--", alpha=0.5)

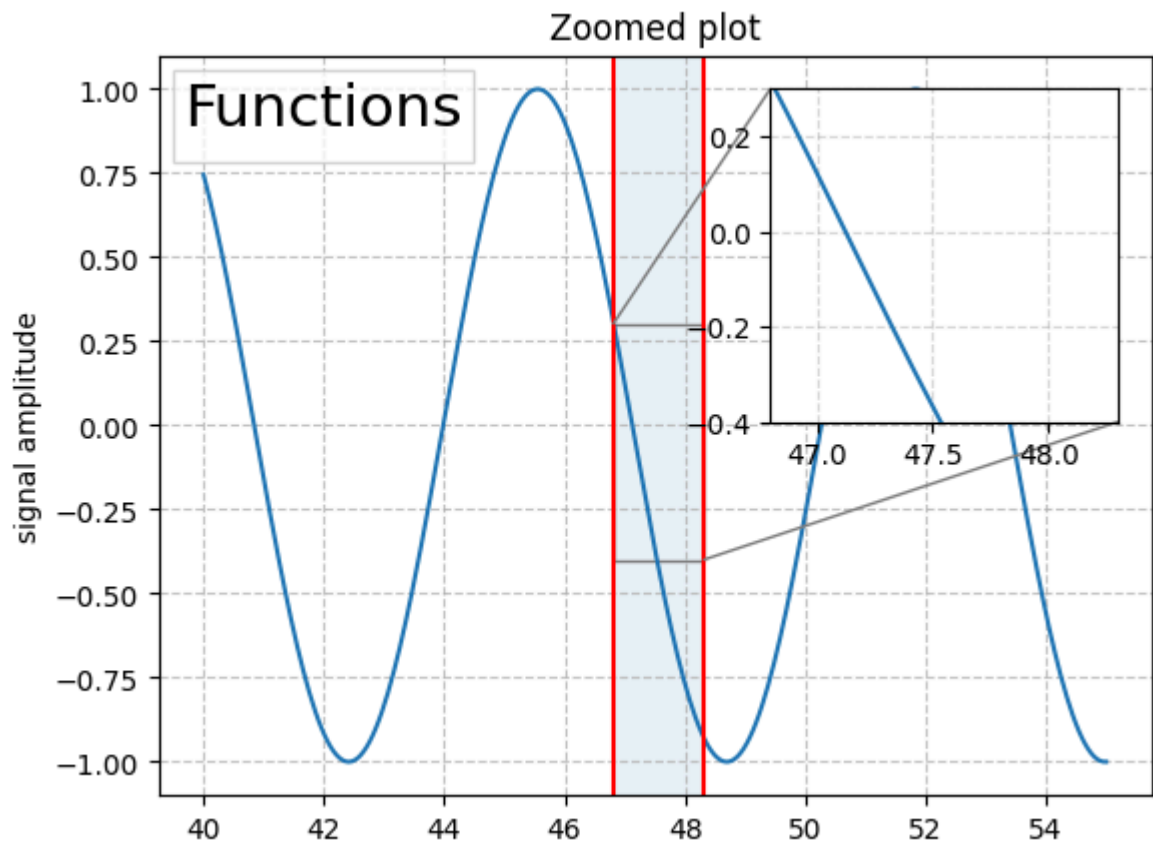
# Draw lines connecting inset to main plot
mark_inset(ax, axins, loc1=2, loc2=4, fc="none", ec="0.5")

ax.legend(
    loc='best',
    bbox_to_anchor=None,
    ncol=1,
    fontsize=10,
    title="Functions",
    title_fontsize=21,
    frameon=True,
    facecolor=None,
    edgecolor=None,
    borderpad=0.4,
    labelspace=0.5,
    handlelength=2.0,
    handleheight=0.7,
    handletextpad=0.8,
    borderaxespac=0.5,
    columnspacing=2.0,
    markerscale=1.0,
    fancybox=False
)

plt.show()

```

C:\Users\KVK\AppData\Local\Temp\ipykernel_3676\1315236393.py:47: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
 ax.legend()



Task 2.12 : Working with date and time data in plots

```
In [64]: import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from datetime import datetime

x = [
    datetime.strptime("2026-02-01", "%Y-%m-%d"),
    datetime.strptime("2026-02-02", "%Y-%m-%d"),
    datetime.strptime("2026-02-03", "%Y-%m-%d")
]

y = [10, 25, 12]

fig, ax = plt.subplots()
ax.plot(x, y)

fig.autofmt_xdate()

ax.xaxis.set_major_formatter(mdates.DateFormatter('%d-%B-%Y'))

now = datetime.now()
timestamp = now.strftime("%Y-%m-%d %H:%M:%S")

ax.text(0.02, 0.95, f"Time: {timestamp}", transform=ax.transAxes)
```

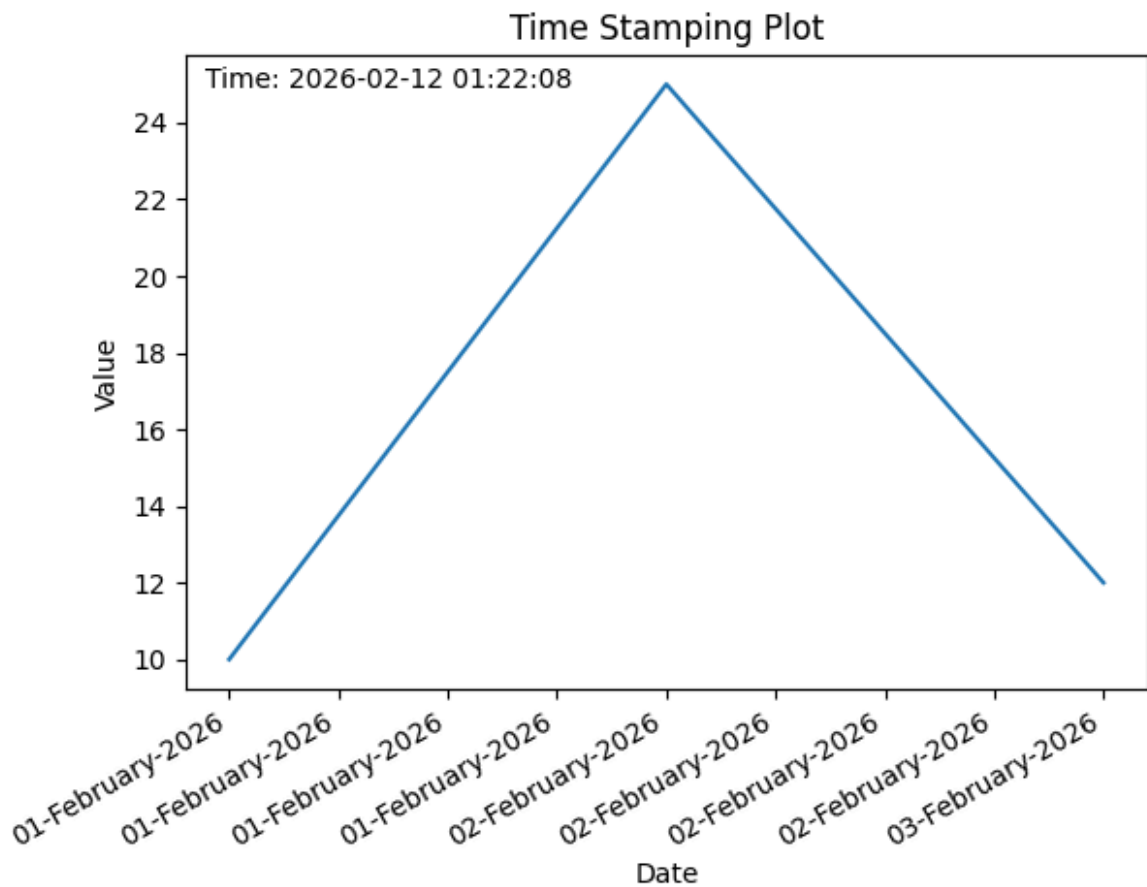


```

ax.set_xlabel("Date")
ax.set_ylabel("Value")
ax.set_title("Time Stamping Plot")

plt.show()

```



Task 3.1: Plot a Pie + Donut Chart

```

In [69]: import matplotlib.pyplot as plt

data = {'M1': 14, 'M2': 21, 'M3': 19, 'M4': 23, 'M5': 23}

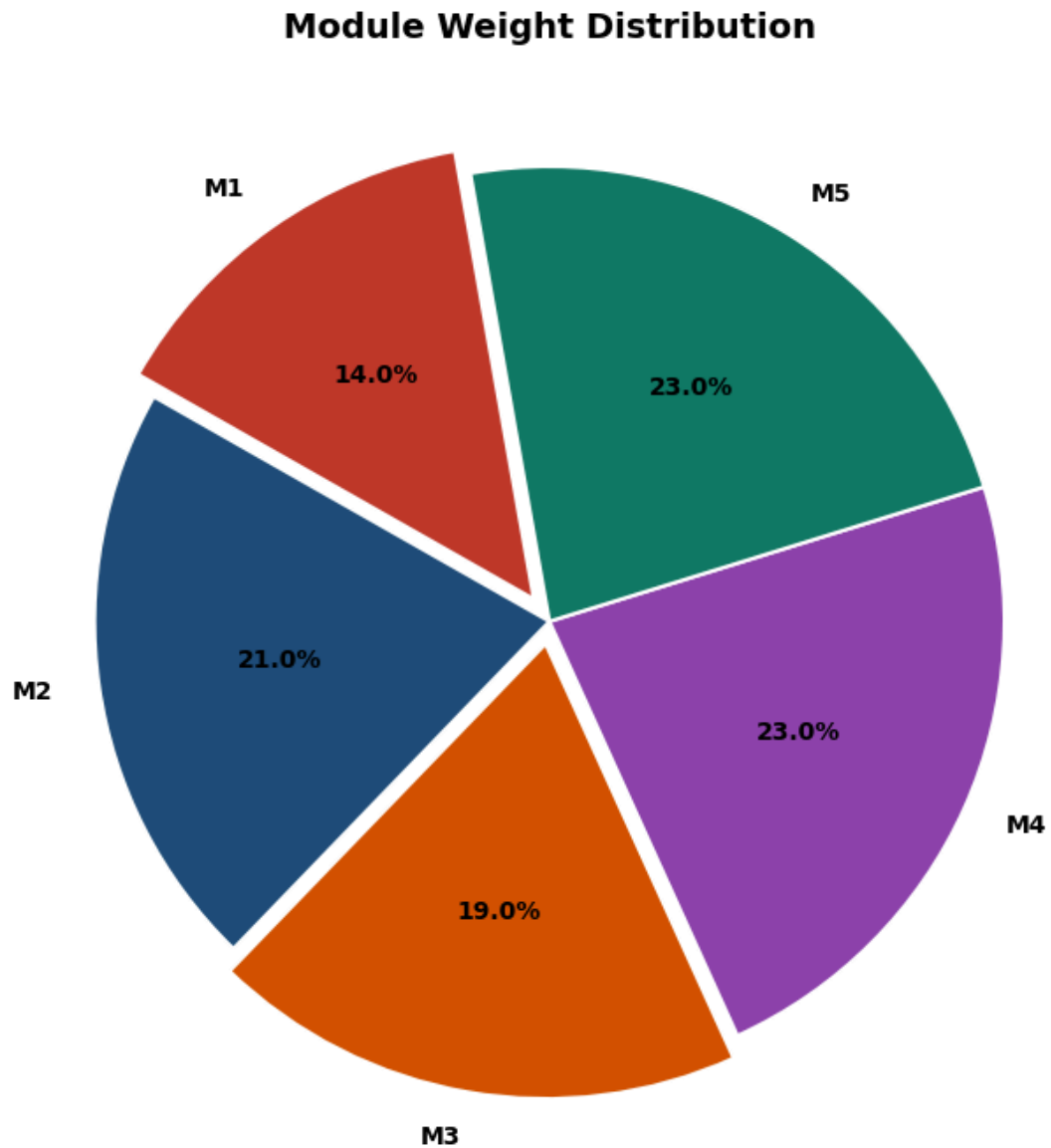
colors = ['#c0392b', # dark red
          '#1f4e79', # dark blue
          '#d35400', # dark orange
          '#8e44ad', # dark purple
          '#117864'] # dark green

plt.figure(figsize=(7,7))

plt.pie(
    data.values(),
    labels=data.keys(),
    colors=colors,
    autopct='%1.1f%%',
    startangle=100,
    explode=[0.06, 0, 0.05, 0, 0],
    textprops={'color':'black', 'weight':'bold'},
    wedgeprops={'edgecolor':'white', 'linewidth':1.5}
)

```

```
plt.title("Module Weight Distribution", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Task 3.2 : Basic PLOtting : Bar (2D & 3D)

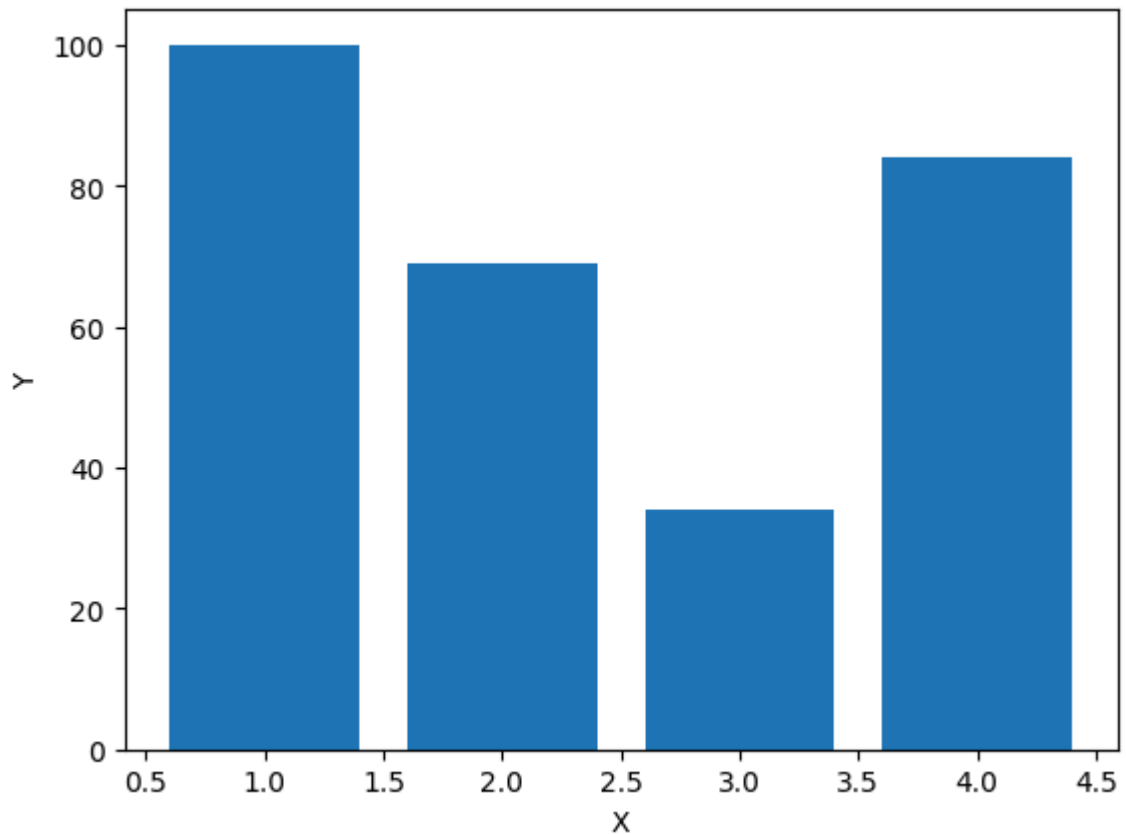
```
In [70]: # 2D bar chart

x = [1,2,3,4]
y = [100, 69, 34, 84]

fig, ax = plt.subplots()

ax.bar(x, y)
ax.set_xlabel("X")
ax.set_ylabel("Y")
```

```
Out[70]: Text(0, 0.5, 'Y')
```



```
In [72]: from mpl_toolkits.mplot3d import Axes3D

x = np.array([1, 2, 3, 4, 5])
y = np.full(len(x), 2)
z_base = np.zeros(len(x))

values = np.array([160, 80, 75, 65, 40])

dx = np.full(len(x), 0.4)
dy = np.full(len(x), 0.4)
dz = values

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

ax.bar3d(x, y, z_base, dx, dy, dz,
         color='royalblue',
         edgecolor='black',
         linewidth=1,
         alpha=0.9)

ax.set_xlabel("X Axis")
ax.set_ylabel("Y Axis")
ax.set_zlabel("Value")

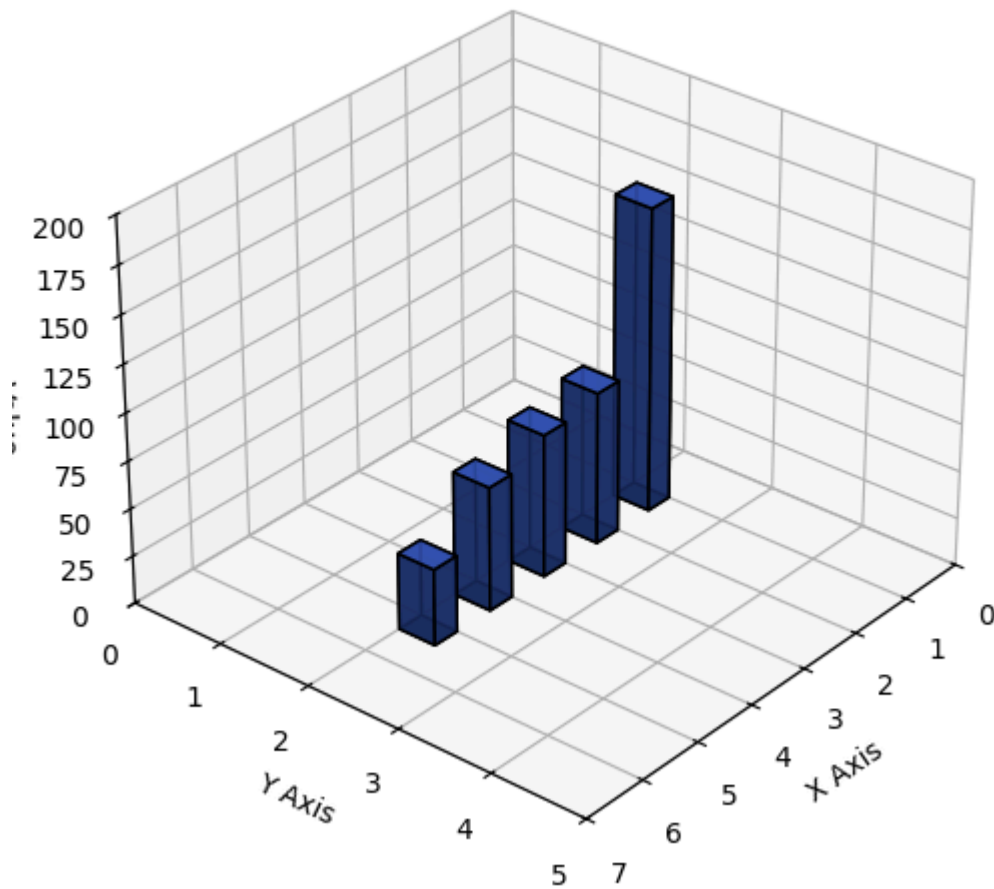
ax.set_xlim(0, 7)
ax.set_ylim(0, 5)
ax.set_zlim(0, 200)

ax.set_title("3D Bar Visualization")

ax.view_init(elev=30, azimuth=40)
```

```
plt.show()
```

3D Bar Visualization



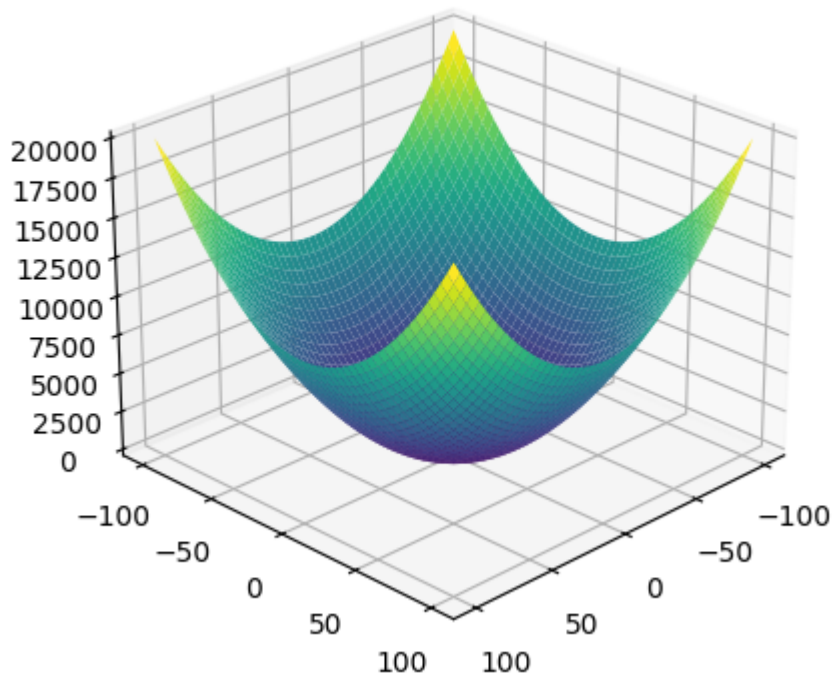
Task 3.3 : $z = x^2 + y^2$

```
In [73]: x = np.linspace(-100, 100)
y = np.linspace(-100, 100)
X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, Y, Z, cmap='viridis')

ax.view_init(elev=25, azimuth=45)
```



Clock animation

```
In [ ]: %matplotlib qt
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from datetime import datetime
from zoneinfo import ZoneInfo
import numpy as np

tz = ZoneInfo("Asia/Kolkata")

def angle_rad(angle, length):
    theta = np.radians(90 - angle)
    return length * np.cos(theta), length * np.sin(theta)

fig, ax = plt.subplots(figsize=(6, 6))

clock_border = plt.Circle((0, 0), 1, fill=False, linewidth=3)
ax.add_artist(clock_border)

for i in range(12):
    angle = i * 30
    x, y = angle_rad(angle, 0.9)
    ax.text(x, y, str(i if i != 0 else 12),
           ha='center', va='center')

center_dot = plt.Circle((0, 0), 0.03)
ax.add_artist(center_dot)

hour_hand, = ax.plot([], [], linewidth=5)
minute_hand, = ax.plot([], [], linewidth=3)
second_hand, = ax.plot([], [], linewidth=1)

ax.set_xlim(-1.1, 1.1)
```

```

ax.set_ylim(-1.1, 1.1)
ax.set_aspect('equal')
ax.axis('off')

title = ax.set_title("")

def update(frame):
    now = datetime.now(tz)

    hour = now.hour % 12
    minute = now.minute
    second = now.second

    hour_angle = (hour + minute / 60) * 30
    minute_angle = (minute + second / 60) * 6
    second_angle = second * 6

    hx, hy = angle_rad(hour_angle, 0.5)
    mx, my = angle_rad(minute_angle, 0.75)
    sx, sy = angle_rad(second_angle, 0.9)

    hour_hand.set_data([0, hx], [0, hy])
    minute_hand.set_data([0, mx], [0, my])
    second_hand.set_data([0, sx], [0, sy])

    title.set_text("Analog Clock - IST " + now.strftime("%H:%M:%S"))

ani = FuncAnimation(fig, update, interval=1000, cache_frame_data=False)

plt.show()

```